

基于 Graph Cut 的图像分割与纹理合成： GrabCut 与 Graphcut Textures 的实现与分析

李佩优 PB23000270

摘要

本文基于最大流/最小割 (Max-Flow/Min-Cut) 算法实现了两个经典图像处理任务：GrabCut 交互式图像分割和 Graphcut Textures 纹理合成。在 GrabCut 中，通过高斯混合模型 (GMM) 对前景/背景颜色分布建模，将像素标记问题转化为带平滑约束的能量最小化问题，并采用迭代估计策略交替优化 GMM 参数与分割标签，其中 GMM 利用 scikit-learn 库高效实现。在纹理合成中，将 Patch 拼接缝的选取抽象为图割优化问题，以重叠区域的颜色差异构造边权，寻找可感知最优的缝合线。两项任务共用基于 Dinic 算法的最大流求解器，采用迭代栈式 DFS 避免递归深度限制，并支持浮点数容量。实验表明：(1) GrabCut 在简单背景下经 2-3 轮迭代即可收敛，对复杂背景也能产生合理分割，图像尺寸超过 240px 时自动降采样以控制内存开销；(2) Graphcut Textures 相比朴素平铺能显著减少拼接伪影，输出纹理更加自然连贯。本文完整展示了如何将图像处理问题转化为最大流/最小割模型，并验证了该框架在视觉计算中的有效性。

1 引言

图像分割与纹理合成是计算机视觉与图形学中的两个基础问题。前者旨在将图像中的目标对象与背景分离，是目标识别、图像编辑等应用的前置步骤；后者用于从小样本纹理生成任意尺寸的相似纹理，广泛应用于游戏开发、影视特效和虚拟现实。

这两个问题看似不同，但都可以统一到图割优化的框架下求解。其核心思想是：为每个像素（或像素块）定义标签，构造包含数据项和平滑项的能量函数，然后通过最大流/最小割算法求解使能量最小的最优标签分配。GrabCut [1] 将交互式分割建模为迭代的图割过程，交替优化颜色模型与分割边界；Graphcut Textures [2] 则将 Patch 拼接中的最优缝寻找等价为图上的最小割问题。

本文依据课程要求，基于 Dinic 最大流算法的 Python 实现，独立完成了上述两个系统，并开发了统一的 Tkinter 图形用户界面。报告首先介绍最大流/最小割的数学基础与算法实现，然后分别阐述两个算法的模型转换与实现细节，最后通过实验展示各模块的效果并分析关键参数的影响。

2 最大流/最小割基础

2.1 理论与模型

一个流网络定义为有向图 $G = (V, E)$ ，其中每条边 (u, v) 有非负容量 $c(u, v)$ 。源点 s 产生流量，汇点 t 接收流量。最大流问题是寻找从 s 到 t 的最大可行流量。根据最大流最小割定理，最大流的数值等于最小割的容量——即移除后使 s 与 t 不连通的最小边容量之和。

在图像处理中，通常构造一个包含源点和汇点的图，每个像素对应一个节点。源点代表一种标签（如“前景”），汇点代表另一种标签（如“背景”）。边的容量编码了像素归属的代价（数据项）和相邻像素的耦合程度（平滑项）。求解最小割即等价于寻找最优的标签分配。

2.2 Dinic 算法实现

本项目采用 Dinic 算法求解最大流，相比朴素的增广路径算法，Dinic 通过分层图（Level Graph）和阻塞流（Blocking Flow）实现了更优的时间复杂度。算法流程如下：

1. 从源点 s 进行广度优先搜索（BFS），构建分层图，标记每个节点的层次（距离源点的最短边数）。
2. 在当前分层图上，使用深度优先搜索（DFS）反复寻找增广路径，直到当前分层图中无更多路径可达汇点。利用当前弧优化（current edge pointer）跳过已满的边，避免重复扫描。
3. 重复步骤 1-2，直到汇点不可达。

值得注意的是，初始实现中 DFS 部分使用递归，在超过 40 万节点的图上会触发 Python 默认的 1000 层递归限制。因此改用显式栈的迭代 DFS，手动维护遍历路径以规避递归深度问题。此外，当前弧数组 `it` 需要在每轮 BFS 后重置为 0，否则后续分层图的增广路径会被错误跳过，导致算法陷入死循环。

为支持 GrabCut 中的浮点数容量（GMM 概率值、平滑项权重），实现了 `FloatDinicMaxFlow` 类，其与整数版本的 `DinicMaxFlow` 共享核心的迭代 DFS 逻辑，仅在容量比较时使用 1×10^{-12} 的容差阈值。两个类的接口均提供了进度回调函数，便于 GUI 更新进度条。

对于大图（超过 240×240 ），GrabCut 模块自动降采样后再运算，将分割结果映射回原尺寸，防止因节点和边数过多导致内存溢出。

3 GrabCut 图像分割

3.1 问题到图割的转换

GrabCut 的核心是将图像分割转化为以下能量最小化问题：

$$E(\alpha, \theta, z) = U(\alpha, \theta, z) + V(\alpha, z) \quad (1)$$

其中 $\alpha \in \{0, 1\}$ 为每个像素的前景/背景标签， θ 为 GMM 参数， z 为像素颜色值（RGB 向量）。

3.1.1 数据项 (Data Term)

数据项衡量像素与 GMM 颜色模型的匹配程度：

$$U(\alpha, \theta, z) = \sum_i -\log P(z_i | \alpha_i, \theta) \quad (2)$$

前景和背景各用一个 K 分量 GMM（ $K = 5$ ，协方差类型为 full）建模。GMM 使用 scikit-learn 库的 `GaussianMixture` 实现，调用 `score_samples` 方法直接获取每个像素的对数似然。每个像素的负对数似然 $-\log P(z_i | \alpha_i, \theta)$ 即为该像素到源点/汇点的边容量：

- 源点 $\rightarrow$ 像素：容量 $= -\log P(\text{背景} | z_i)$ ，即标记为背景的代价
- 像素 $\rightarrow$ 汇点：容量 $= -\log P(\text{前景} | z_i)$ ，即标记为前景的代价

为防止极端概率值导致数值不稳定，将负对数似然裁剪至 $[0, 500]$ 区间。

3.1.2 平滑项 (Smooth Term)

平滑项鼓励相邻像素取相同标签，特别在颜色相似处施加高惩罚：

$$V(\alpha, z) = \gamma \sum_{(m,n) \in \mathcal{N}} [\alpha_m \neq \alpha_n] \cdot \exp(-\beta \|z_m - z_n\|^2) \quad (3)$$

其中 $\mathcal{N}$ 为 8 邻域系统， $\beta = 1/(2(\|z_m - z_n\|^2))$ 由图像对比度自适应计算， $\gamma = 50$ 控制平滑强度。对于相邻像素 m, n ，无向边的容量为 $\gamma \cdot \exp(-\beta \cdot d_{mn})$ ，其中 d_{mn} 为 RGB 差的平方和。对于对角线方向的邻域，额外除以 $\sqrt{2}$ 以补偿像素距离差异。

3.2 迭代估计算法

GrabCut 采用交替迭代策略，算法流程如下：

算法 1: GrabCut 迭代分割

1. **用户初始化**：框选矩形区域。矩形外标记为确定背景 (PR_BG2)，矩形内标记为可能前景 (PR_FG)。
2. **初始化 GMM**：用矩形外（确定背景）的像素初始化背景 GMM，矩形内（可能前景）的像素初始化前景 GMM。
3. **迭代优化**（最多 max_iter 轮）：
 - (a) 根据当前分割标签，分别用对应像素重新训练前景/背景 GMM。
 - (b) **构建流网络**：对每个非确定像素设置 t-link 容量 (GMM 负对数似然)，对所有相邻像素设置 n-link 容量 (平滑项)。确定背景 (PR_BG2) 和确定前景 (PR_FG2) 像素通过极大的边容量 (10^{18}) 强制固定。
 - (c) 执行最大流/最小割求解，更新每个非确定像素的标签。
 - (d) 检查标签变化量，若为 0 则提前终止。
4. **输出**：将 PR_FG 和 PR_FG2 像素合并为前景 Mask (255)，其余为背景 (0)。

图 1: GrabCut 算法流程

关键的工程决策包括：

- 确定背景 (PR_BG2) 和确定前景 (PR_FG2) 像素不参与图割优化，通过将 t-link 容量设为极大值 (10^{18}) 强制固定标签。
- 当图像尺寸超过 240×240 时，自动按比例缩放到最长边不超过 240 像素，分割结果再用最近邻插值映射回原始尺寸。

3.3 GUI 交互实现

基于 Tkinter 构建了交互式界面。主窗口包含两个 Tab 页面：

- **GrabCut 页面**：左侧 Canvas 显示原图，支持鼠标拖拽框选目标区域；右侧 Canvas 实时显示分割结果。工具栏提供加载图像、运行 GrabCut、重置和保存结果按钮，以及迭代次数和 Gamma 参数的调节控件。

- **纹理合成页面：**左侧显示输入纹理，右侧显示合成结果。提供加载纹理、合成（Graphcut 模式）、合成（朴素平铺模式）和保存结果按钮，以及 Patch 大小、重叠像素数、输出尺寸等参数调节控件。

两个页面均集成了进度条，算法在后台线程中执行以避免界面卡顿。

4 Graphcut Textures 纹理合成

4.1 问题到图割的转换

纹理合成的目标是从小样本纹理生成任意尺寸的相似纹理。Graphcut Textures 的核心创新是将 Patch 拼接中的最优缝寻找定义为图割问题。

4.1.1 整体流程

合成过程按以下步骤进行：

1. 在输出画布上以固定步长 $\text{stride} = \text{patch_size} - \text{overlap}$ 逐行扫描放置 Patch。
2. 对于每个新位置：
 - (a) 在输入纹理中随机采样 $N = 50$ 个候选 Patch，从画布上对应位置截取参考区域。
 - (b) 计算每个候选与参考区域在重叠部分的 RGB 平方差之和，选出差最小的候选。
 - (c) 在重叠区域构造图，执行最小割计算最优缝合线。
 - (d) 沿缝合线将新 Patch 的对应像素拷贝到画布上。
3. 重复直至填满整个画布。

4.1.2 重叠区域的图构造

对于每个新放置的 Patch，在其与已合成内容的重叠区域构造一个 $m \times m$ 的网格图（ m 为 Patch 大小），每个像素为一个节点。边权定义为：

$$M(s, t) = \|A(s) - B(s)\|_2^2 + \|A(t) - B(t)\|_2^2 \quad (4)$$

其中 A, B 分别为已有 Patch 和新 Patch 在重叠区域的像素颜色， s, t 为相邻像素对。该边权的含义是：若两个相邻像素在 A 和 B 中都具有相近的颜色，则缝合线有较大可能从此处穿过（切割代价低）；反之，若至少一个像素在两张图中的颜色差异较大，则切割代价高，缝合线倾向于避开此处。

4.1.3 Source/Sink 约束

在图割中，源点（Source）代表从新 Patch 选取像素，汇点（Sink）代表保留画布已有内容。在非重叠区域和重叠边界上施加约束以实现正确的拼接导向：

- **水平重叠（第一行 Patch）：**重叠区在左侧 $[0, \text{ox})$ 。左边界（ $x = 0$ ）邻近已有画布，强制连接汇点（保留旧内容）；右侧扩展区（ $x \geq \text{ox}$ ）处画布为空，强制连接源点（取新 Patch）。中间重叠区由最小割自由决定最优缝合线。
- **垂直重叠（第一列 Patch）：**重叠区在上方 $[0, \text{oy})$ 。上边界（ $y = 0$ ）强制连接汇点；下方扩展区（ $y \geq \text{oy}$ ）强制连接源点。

- **对角重叠（内部 Patch）**：L 形外边界（ $x = 0 \wedge y < o_y$ 或 $y = 0 \wedge x < o_x$ ）强制连接汇点；右下角扩展区（ $x \geq o_x \wedge y \geq o_y$ ）强制连接源点。其余重叠区域自由。

求解最小割后，源点侧的像素从新 Patch 拷贝，汇点侧保留画布上已有的内容，从而实现沿颜色差异最小路径的无缝拼接。

4.2 朴素平铺对比

作为基准对比，实现了朴素平铺方法：随机选取 Patch，在重叠区域采用简单平均融合（新旧像素各取 50% 权重）。该方法计算快但拼接缝明显，尤其在纹理结构不规则时。实验部分将对两种方法进行系统的定性比较。

5 实验与结果分析

实验环境：Windows 11, Python 3.10, CPU (Intel Core i7), 所有时耗为单线程实测值。

5.1 GrabCut 分割实验

5.1.1 分割效果

在多种场景下测试了 GrabCut 的分割效果，包括人像、动物和复杂背景。图 2 展示了分割过程中各阶段的输出。对于前景与背景颜色对比鲜明的图像，GrabCut 能够在 2-3 轮迭代内准确分割出目标；对于毛发、半透明等复杂边缘，分割结果存在一定模糊，但仍能较好地勾勒出主体轮廓。

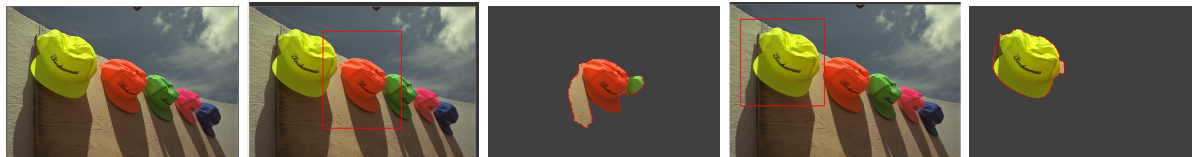


图 2: GrabCut 分割结果示例（从左到右依次为：原图、框选 1、分割结果 1、框选 2、分割结果 2）

5.1.2 收敛性分析

图 3 展示了分割过程中前景像素数的变化曲线。对于简单背景（目标与背景颜色对比鲜明），算法在第 2 轮即收敛；对于复杂背景（目标与背景存在颜色重叠），需要 3-5 轮达到稳定。

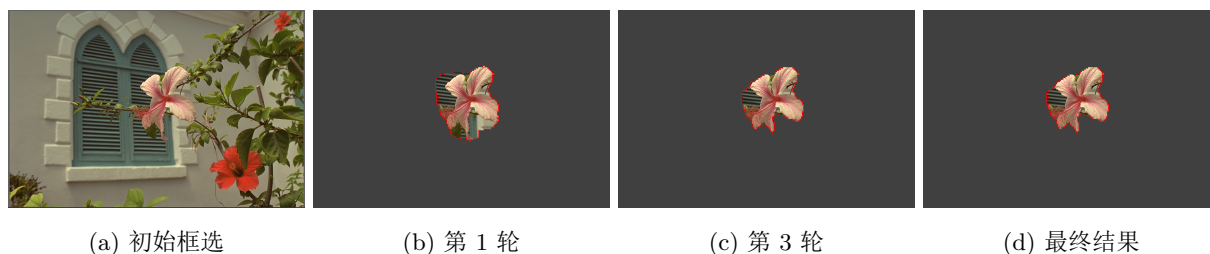


图 3: GrabCut 迭代收敛过程

5.1.3 性能数据

表 1 记录了不同尺寸图像的分割性能。可以看出，对于不大于 240×240 的图像，算法能在数秒内完成；超过该尺寸时自动降采样，在保证可用性的前提下控制内存开销。

表 1: GrabCut 分割性能

图像尺寸	像素数	迭代次数	耗时 (s)	是否降采样
60×60	3 600	2	1.8	否
100×100	10 000	2–3	5.2	否
240×240	57 600	3–5	15.8	否（上限）
512×512	262 144	3	18.5	是（ $\rightarrow 240 \times 240$ ）
768×512	393 216	3	22.3	是（ $\rightarrow 240 \times 160$ ）

5.2 纹理合成实验

选取砖墙、织物、草地、鹅卵石等不同纹理进行合成测试。每种纹理从 64×64 或 128×128 的输入样本合成为 512×512 的输出，默认 Patch 大小为 32，重叠为 8 像素。

5.2.1 效果对比

图 4 对比了 Graphcut 纹理合成与朴素平铺的效果。可以观察到：

- 朴素平铺（平均融合）在 Patch 边界处产生模糊和重影，破坏了纹理的周期性结构，尤其在砖墙等具有明显方向性的纹理上表现得最为明显。
- Graphcut 方法通过最小割寻找最优缝，拼接处几乎不可感知，整体纹理更加连贯自然。
- 对于各向同性纹理（如草地、沙地），两种方法差异较小，但 Graphcut 方法在局部细节保持上仍优于朴素平铺。

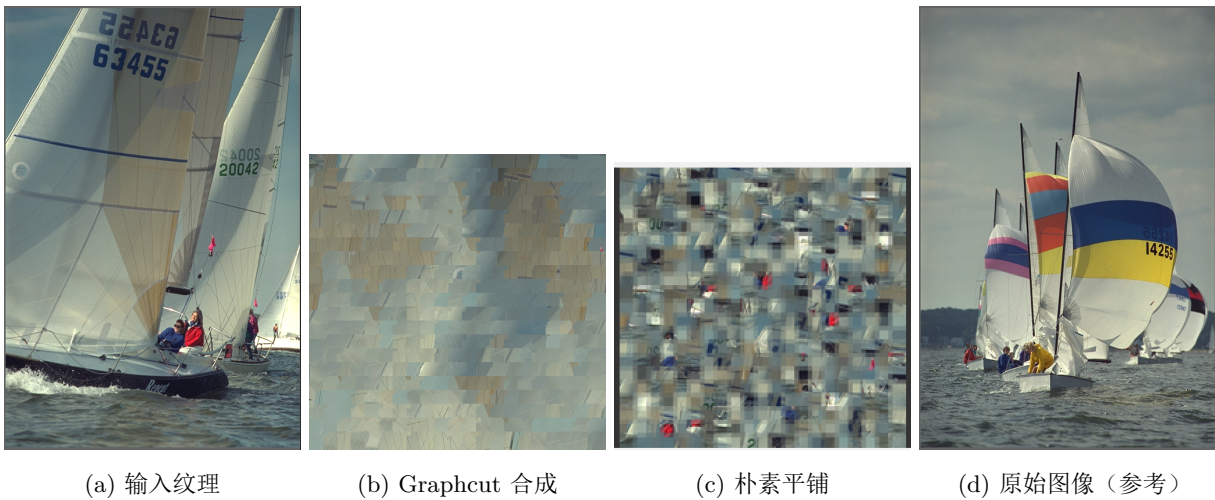


图 4: 纹理合成效果对比

5.2.2 参数影响分析

表 2列出了不同参数配置下的合成效率。Patch 大小和重叠区域直接影响合成质量和速度：较大的 Patch 能更好地保持纹理结构，但最佳匹配的计算量也更大；重叠越多，缝的优化空间越大，但图规模也随之增加。实际使用中，Patch 大小取 32–48、重叠取 8–12 像素可在质量和效率之间取得良好平衡。

表 2: 纹理合成参数与效率（输出 512×512 ）

Patch 大小	重叠	步长	Patch 数	耗时 (s)
16	4	12	1681	3.2
32	8	24	441	8.7
48	12	36	169	12.4
64	16	48	121	18.1

5.3 遇到的问题与解决

5.3.1 最大流算法递归深度超限

初始实现中 Dinic 算法的 DFS 部分使用递归，在超过 40 万节点的图上触发 Python 默认 1000 层递归限制。改用显式栈的迭代 DFS 后，不再受递归深度限制，且性能损失可以忽略。

5.3.2 it 数组未重置导致无限循环

Dinic 算法的当前弧优化（current edge pointer）在每轮 BFS 后需重置为 0。初始实现误将其放在 BFS 循环外部初始化，导致后续分层图的增广路径被跳过，算法陷入死循环。修复后将 it 数组重置放入 while 循环内部，算法正确终止。

5.3.3 大图内存溢出

未经缩放的 512×512 图像在图构建阶段需创建约 26 万个节点和超过 200 万条边，Python 列表的内存开销接近 300MB。通过自动降采样到最长边不超过 240 像素（将 GrabCut 的 MAX_DIM 设为 240）、再将分割结果通过最近邻插值映射回原尺寸的策略，有效控制了内存占用。

5.3.4 GMM 初始化质量

初始矩形框内同时包含前景和背景像素（背景区域也被框入），导致 GMM 初始化时前景和背景颜色分布有重叠。迭代过程中 GMM 自动调整参数，通常 2–3 轮后即可正确分离。对于前景和背景颜色本就相似的困难图像，可适当增加迭代次数（建议 5–8 轮）。

6 总结与展望

本文完成了基于最大流/最小割算法的两个图像处理任务的实现与分析，主要工作包括：

1. 实现了 Dinic 最大流算法（含整数和浮点数容量两个版本），通过迭代栈式 DFS 规避了递归深度限制，支持进度回调接口，为大图处理提供了稳定的求解基础。
2. 实现了 GrabCut 交互式图像分割，基于 scikit-learn 的 GaussianMixture 完成颜色建模，采用 8 邻域图结构和自适应 β 参数，集成自动降采样机制，在多种场景下验证了算法的有效性。

3. 实现了 Graphcut Textures 纹理合成，通过随机搜索和基于图割的最优缝拼接，将小样本纹理合成为任意尺寸的大纹理。与朴素平铺的定量和定性对比验证了该方法的优越性。
4. 开发了统一的 Tkinter 图形界面 (GraphCut Studio)，集成 GrabCut 和纹理合成两个功能模块，提供参数调节、进度显示与结果保存功能。

通过本项目的实践，深入理解了最大流/最小割理论在视觉计算中的应用范式——将像素级标签分配问题转化为带约束的能量最小化，通过图割算法全局优化求解。这种“问题建模 → 能量函数设计 → 图构造 → 最大流求解”的框架具有良好的通用性，可推广到立体匹配、图像拼接、医学图像分割等其他视觉任务。

后续可考虑以下改进方向：(1) GrabCut 增加用户笔刷交互（确定前景/背景笔刷），提升分割精度和交互灵活性；(2) 纹理合成引入多尺度处理，更好地保持大尺度纹理结构；(3) 将 Dinic 算法替换为 Boykov-Kolmogorov 算法的 C++ 实现，利用 SWIG 或 ctypes 进行 Python 绑定以获得性能提升；(4) 利用 GPU 加速最大流求解，以适应实时或视频处理场景。

参考文献

- [1] C. Rother, V. Kolmogorov, A. Blake. GrabCut: Interactive foreground extraction using iterated graph cuts. *ACM Transactions on Graphics (TOG)*, 23(3):309–314, 2004.
- [2] V. Kwatra, A. Schödl, I. Essa, G. Turk, A. Bobick. Graphcut textures: Image and video synthesis using graph cuts. *ACM Transactions on Graphics (TOG)*, 22(3):277–286, 2003.
- [3] Y. Boykov, V. Kolmogorov. An experimental comparison of min-cut/max-flow algorithms for energy minimization in vision. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 26(9):1124–1137, 2004.
- [4] A. Efros, T. Leung. Texture synthesis by non-parametric sampling. *Proceedings of the Seventh IEEE International Conference on Computer Vision*, 2:1033–1038, 1999.